BMH309 Mobil Cihaz Programlama

Dart ile karar verme, tekrar ve güvenli kullanıcı girdisi

Dr. Öğr. Üyesi Fesih Keskin

Bilgisayar Mühendisliği

Ders formatı: Slayt → VS Code → Dönüş ve tartışma

Bu haftanın öğrenme hedefleri

- `if / else if / else`, üçlü operatör ve `switch` ile karar vermek.
- Operatörleri, önceliği ve kısa devre değerlendirmeyi açıklamak.
- `for`, `for-in`, `while`, `do-while` döngülerini seçmek.
- `break`, `continue` ve etiketlerle akışı yönetmek.
- `stdin`, `tryParse` ve girdi doğrulama kullanmak.
- Konuları terminalde çalışan küçük bir uygulamada birleştirmek.

Dersi nasıl işleyeceğiz?

1. Slayt	2. VS Code	3. Dönüş
Kavram	Uygulama	Tartışma
Kural, şekil veya tahmin sorusu	Dosyayı yaz, çalıştır, değiştir	Çıktı, hata ve alternatif çözüm

Sunumun rolü

Slaytta uzun programları okumayacağız. **Tam kodlar ders notunda**; uzun kodları VS Code'da birlikte yazacağız. Ardından **Dönüş** slaytına gelip çıktıları, hataları ve alternatif çözümleri tartışacağız.

Sunum ile ders notu nasıl eşleşir?

Sunumda	**Ders notunda**
Başlıktaki numara: **3.5** | **else if**	Bölüm **3.5**
VS Code | **3.4–3.6** slaytındaki dosya adları	Aynı adlı kod blokları (`03_08_else_if.dart` gibi)
Dönüş | **3.5** slaytındaki çıktılar	Kodun hemen altındaki **Çıktı** bloğu
Alt bilgi çubuğu: *Ders notu: Bölüm 3 ...*	O an işlenen bölüm

Kavram slaytı

Başlık bölüm numarasıyla başlar.

VS Code slaytı

Hangi dosyalar, hangi sırayla?

Dönüş slaytı

Önce tahmin, sonra çıktı.

Yol haritası ve önerilen zaman planı

Blok	Ders notu bölümleri
Blok 1 (50 dk)	**0** Başlamadan önce · **1** Program akışı · **2** Ön bilgi · **3** if / else if / else · **4** Üçlü operatör
Blok 2 (50 dk)	**5** switch · **6–7** Operatörler · **8** for · **9** while
Blok 3 (50 dk)	**10** do-while · **11** break / continue / label · **12** Kullanıcıdan veri · **13** Cep Cüzdan · **14** Not problemi
Evde	**15** Özet · **16** Kendinizi test edin · **17** Ödev 1 · **18** Kaynaklar

Zaman daralırsa

Hızlı geçilebilecekler: **5.6** when, **7.5** bit operatörleri, **12.8** UTF-8 girdi. Bölüm **14**'ün çözümleri evde de çalışılabilir. Hepsi ders notunda eksiksiz anlatılıyor.

Çalışma ilkeleri | AI Seviye 0

- Örnekleri ve ödevi **yapay zekâ kullanmadan** çözün.
- Copilot ve benzeri otomatik kod tamamlama araçlarını kapatın.
- Kopyala-yapıştır mümkün olsa da önce çıktıyı tahmin edin.
- Her örnekte en az bir değişkeni değiştirip yeniden çalıştırın.
- Çalışan kod kadar **hata mesajını okumak** da öğrenme hedefidir.

0 | Başlamadan Önce

VS Code'da çalışma düzeni

Ders notu: Bölüm 0 · Dosyalar: –

0.1–0.2 | Çalışma klasörü ve çalıştırma

1. `bmh309/hafta2/` klasörünü VS Code'da açın.
2. Her kod bloğunun ilk satırı dosya adını verir: `// Dosya: 03_01_if_temel.dart`
3. Dosyayı o adla oluşturun, bloğun **tamamını** yapıştırın, kaydedin.

Yol	**Nasıl?**	**Çıktı nerede?**
Run düğmesi	`void main()` üstündeki **Run** veya `F5`	Debug Console
Terminal	`Ctrl + ‘` → `dart run 03_01_if_temel.dart`	Terminal

0.3 | Önemli teknik ayar: terminal girişi

Klavye girdisi alan örnekler

`12_...`, `13_...` ve `14_...` dosyalarını **terminalden** çalıştıracağız: `dart run dosya.dart`

- Debug Console, `stdin.readLineSync()` için uygun değildir.
- Alternatif VS Code ayarı: `"dart.cliConsole": "terminal"`
- Türkçe karakter sorunu yaşanırsa Windows terminalinde `chcp 65001`.

0.4–0.5 | Hata türleri ve kısayollar

Ders notundaki hatalı örnekler bilerek hatalıdır

Derleme hatası: program başlamaz, satır kırmızıyla çizilir.
Çalışma zamanı hatası: program çalışırken durur (`Unhandled exception`).
Mantık hatası: program çalışır ama **yanlış sonuç** üretir. En tehlikelisi budur.

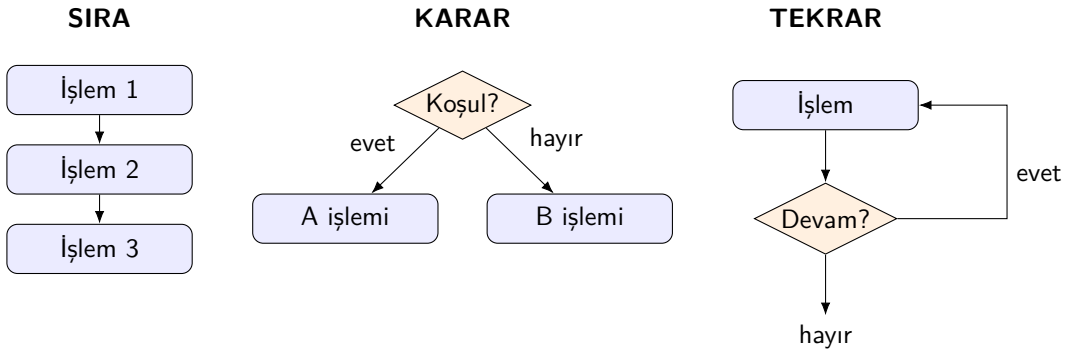
Biçimlendir	`Shift + Alt + F`
Problems paneli	`Ctrl + Shift + M`
Programı durdur	Terminalde `Ctrl + C`, VS Code'da kırmızı kare

1 | Program Akışı

Sıra · Karar · Tekrar

Ders notu: Bölüm 1 · Dosyalar: –

1 | Bir programın üç temel akış modeli



Flutter bağlantısı

Ekranda hata / yükleme / içerik **seçmek** karardır; listedeki her ürün için kart **oluşturmak** tekrardır.

2 | Ön Bilgi

Karşılaştırma ve mantıksal operatörler

Ders notu: Bölüm 2 · Dosyalar: 02_01

Karşılaştırmalar

`==`	eşit mi?
`!=`	farklı mı?
`<` / `>`	küçük / büyük mü?
`<=` / `>=`	sınır dahil mi?

Mantıksal bağlaçlar

`&&`	VE
`||`	VEYA
`!`	DEĞİL

Düşünelim

`pil = 18` ve `sarjda = false` iken `pil < 20 && !sarjda` sonucu nedir?

VS Code | 2 · İlk bool ifadeler

Dosya

`02_01_on_bilgi.dart` **Birlikte yazalım**

1. `pil = 18`, `sarjda = false` değişkenlerini tanımlayın.
2. `==`, `!=`, `&&`, `||` için sonuçları ekrana yazdırın.
3. `sarjda = true` yapıp hangi çıktıların değiştiğini gözleyin.

Sorgu

= atama yaparken == neden bir `bool` üretir?

02_01_on_bilgi.dart *tahmin ettiniz mi?*

02_01_on_bilgi.dart *tahmin ettiniz mi?*

```
true
false
true
true
true
```

02_01_on_bilgi.dart *tahmin ettiniz mi?*

```
true
false
true
true
true
```

sarjda = true olunca

Yalnızca 4. satır değişir: `pil < 20 && !sarjda` artık `false`. VEYA (`||`) satırı yine `true`.

= ile ==

= bir değeri **atar**. == iki değeri **karşılaştırır** ve sonucu `true` / `false` olur.

3 | if, else if ve else

Koşullara göre hangi blok çalışacak?

Ders notu: Bölüm 3 · Dosyalar: `03_01 – 03_18`

3.1 | if: koşul doğruysa blok çalışır

Kalıp

```
if (kosul) {
    // yalnızca kosul true ise
}
```

- Koşul `bool` olmalıdır.
- `( )`: koşul.
- `{ }`: çalışacak blok.
- Blok dışındaki satırlar normal akışta devam eder.

3.2 | Tuzak: girinti blok oluşturmaz

Bu kod ne yazdırır? (`bildirim = 0`)

```
    if (bildirim > 0)
    print('Yeni bildiriminiz var.');
    print('Bildirim simgesi gösterildi.');
```

3.2 | Tuzak: girinti blok oluşturmaz

Bu kod ne yazdırır? (`bildirim = 0`)

```
if (bildirim > 0)
print('Yeni bildiriminiz var.');
print('Bildirim simgesi gösterildi.');
```

Dart'ta girinti değil, süslü parantez belirleyicidir. İkinci `print` her durumda çalışır. Tek satırlık `if`'i aynı satıra yazın; birden fazla satır varsa **her zaman** `{ }` kullanın.

3.2 | Tuzak: if sonundaki noktalı virgül

Mantık hatası

```
int pil = 80;
if (pil < 20); {
  print('Pil düşük!');
}
```

3.2 | Tuzak: if sonundaki noktalı virgül

Mantık hatası

```
int pil = 80;
if (pil < 20); {
  print('Pil düşük!');
}
```

- `if (...);` ifadesi **boş bir işlem** ile biter.
- Sonraki `{ ... }` bağımsız bir bloktur; koşuldan bağımsız çalışır.
- **Kural:** `if (kosul)` satırının sonuna `;` koymayın.

3.3 | Koşul mutlaka bool olmalıdır

Python / JavaScript alışkanlığı

```
int bildirim = 3;
if (bildirim) {     // HATA
    print('Bildirim var');
  }
```

Dart'ta doğrusu

```
if (bildirim > 0) { ... }
if (mesaj.isEmpty) { ... }
```

Dart'ta sıfır olmayan sayılar veya dolu metinler kendiliğinden "doğru" sayılmaz. Koşul `true` ya da `false` üreten bir ifade olmalıdır.

VS Code | 3.1–3.3 · İlk if ve bilinçli hatalar

Dosyalar

`03_01_if_temel.dart` · `03_02_tek_satir_if.dart`	**Birlikte yazalım**
`03_03_tuzak_parantez.dart` · `03_04_tuzak_noktali_virgul.dart`	**Önce tahmin**
`03_05_tuzak_bool.dart` · `03_06_bool_dogru.dart`	**Hata mesajını oku**

1. Pil seviyesi 15 iken uyarıyı çalıştırın; sonra 50 yapın.
2. İki tuzak dosyasında sonucu **çalıştırmadan önce** tahmin edin.
3. `if (bildirim)` satırının hata metnini Problems panelinde okuyun ve düzeltin.

Sorgu

Mantık hatası mı, derleme hatası mı? Ayrımı neye göre yapıyoruz?

03_01 (`pil = 15`)

```
Pil seviyesi düşük, şarj cihazını takın.
Program bitti.
```

03_01 (`pil = 50`) → yalnızca `Program bitti.`

Dönüş | 3.1–3.3 · Çıktılar ve hatalar

03_01 (`pil = 15`)

```
Pil seviyesi düşük, şarj cihazını takın.
Program bitti.
```

03_01 (`pil = 50`) → yalnızca `Program bitti.`

03_03 tuzak parantez (`bildirim = 0`)

```
Bildirim simgesi gösterildi.
```

03_04 tuzak noktalı virgül (`pil = 80`)

```
Pil düşük!
```

Dönüş | 3.1–3.3 · Çıktılar ve hatalar

03_01 (`pil = 15`)

```
Pil seviyesi düşük, şarj cihazını takın.
Program bitti.
```

03_01 (`pil = 50`) → yalnızca `Program bitti.`

03_03 tuzak parantez (`bildirim = 0`)

```
Bildirim simgesi gösterildi.
```

03_04 tuzak noktalı virgül (`pil = 80`)

```
Pil düşük!
```

03_05 derleme hatası

```
Conditions must have a static type of 'bool'.
```

Tartışma

03_03 ve 03_04 **mantık hatasıdır**: program çalışır, sonuç yanlıştır. 03_05 **derleme hatasıdır**: program hiç başlamaz. Dart'ın derleme aşamasında yakaladığı hata, kullanıcının telefonuna hiç ulaşmaz.

3.4–3.5 | else ve else if: hangi yol seçilir?

İki olasılık (3.4)

`if (internetVar)` → verileri yükle
`else` → çevrimdışı mesajı

Üç ve daha fazla olasılık (3.5)

```
if (sarjOluyor)
else if (pilSeviyesi <= 5)
else if (pilSeviyesi <= 20)
else
```

En kritik kural

`else if` zincirinde koşullar yukarıdan aşağıya denenir. **İlk doğru kol çalışır ve zincir biter.**

3.5 | Koşul sırası: özelden genele

Hangi mesaj görünür? (03_09_tuzak_sira.dart)

```
int pilSeviyesi = 3;
if (pilSeviyesi <= 20) {
    print('Düşük pil');
  } else if (pilSeviyesi <= 5) {
    print('KRİTİK pil!');
  }
```

3.5 | Koşul sırası: özelden genele

Hangi mesaj görünür? (03_09_tuzak_sira.dart)

```dart
int pilSeviyesi = 3;
if (pilSeviyesi <= 20) {
    print('Düşük pil');
  } else if (pilSeviyesi <= 5) {
    print('KRİTİK pil!');
  }
```

Yanıt: `Düşük pil`. Kritik kola **hiçbir zaman** ulaşılamaz; önce `<= 5`, sonra `<= 20` yazılmalıdır. Derleyici bu hatayı fark etmez.

3.5 | Ayrı if mi? else if zinciri mi?

Bağımsız if'ler

```
puan = 95
if (puan >= 90) → Harika
if (puan >= 70) → İyi
if (puan >= 50) → Geçti
```

else if zinciri

Aynı koşullar zincirde olsaydı

yalnızca "Harika!" yazılırdı.

Birden fazla sonuç aynı anda geçerli olabiliyorsa **ayrı if**, seçeneklerden yalnızca biri seçilecekse **else if**.

3.6 | Birleşik koşullar: form doğrulama

Kontrol sırası: boş → biçim → uzunluk

`eposta.isEmpty || sifre.isEmpty` → "Lütfen tüm alanları doldurun."

`!eposta.contains('@')` → "Geçerli bir e-posta adresi girin."

`sifre.length < 8` → "Şifre en az 8 karakter olmalı."

`else` → "Giriş yapılıyor..."

Flutter bağlantısı

Form alanlarının doğrulayıcıları (`validator`) bu mantıkla yazılır. 3. haftada bu kodu bir fonksiyona dönüştüreceğiz.

VS Code | 3.4–3.6 · Pil durumu ve form doğrulama

Dosyalar

`03_07_else.dart` · `03_08_else_if.dart`	**Birlikte yazalım**
`03_09_tuzak_sira.dart` · `03_10_ayri_if_zincir.dart`	**Önce tahmin**
`03_11_giris_dogrulama.dart`	**Birlikte yazalım**

1. Pil için 18, 3, 80 ve "şarjda + 3" senaryolarını sırayla deneyin.
2. Koşulların sırasını değiştirince ne olduğunu inceleyin.
3. E-posta ve şifre değerlerini değiştirerek dört farklı mesajı da elde edin.

Sorgu

Bir kullanıcı aynı anda iki hata yaptıysa hangi uyarıyı önce göstermeliyiz?

03_08_else_if.dart *pil, şarj → çıktı*

03_08_else_if.dart *pil, şarj → çıktı*

```
 18, false → Düşük pil: Güç tasarrufu modu ö
nerilir.
 3, false → KRİTİK! Cihaz kapanmak üzere.
 80, false → Pil durumu iyi (%80)
 3, true  → Şarj oluyor (%3)
```

Neden şarjdayken kritik uyarı yok?

İlk kol `sarjOluyor` doğru; zincir orada biter.

03_08_else_if.dart *pil, şarj → çıktı*

```
 18, false → Düşük pil: Güç tasarrufu modu ö
nerilir.
 3, false → KRİTİK! Cihaz kapanmak üzere.
 80, false → Pil durumu iyi (%80)
 3, true  → Şarj oluyor (%3)
```

03_10 ayrı if ile zincir (`puan = 95`)

```
--- Ayrı if blokları ---
Harika!
İyi!
Geçti.
--- else if zinciri ---
Harika!
```

Neden şarjdayken kritik uyarı yok?

İlk kol `sarjOluyor` doğru; zincir orada biter.

03_11_giris_dogrulama.dart *eposta, şifre → çıktı*

03_11_giris_dogrulama.dart *eposta, şifre → çıktı*

```
''              , '12345'     → Lütfen tüm alanları doldurun.
'ayse'          , '12345678'  → Geçerli bir e-posta adresi girin.
'ayse@ornek.com', '12345'     → Şifre en az 8 karakter olmalı.
'ayse@ornek.com', 'gizli2026' → Giriş yapılıyor...
```

03_11_giris_dogrulama.dart *eposta, sifre → çıktı*

```
''              , '12345'     → Lütfen tüm alanları doldurun.
'ayse'          , '12345678'  → Geçerli bir e-posta adresi girin.
'ayse@ornek.com', '12345'     → Şifre en az 8 karakter olmalı.
'ayse@ornek.com', 'gizli2026' → Giriş yapılıyor...
```

Alternatif tartışma

Kontrolleri **ayrı if'ler** olarak yazsaydık kullanıcı aynı anda birden fazla uyarı görürdü. Hangisi daha iyi bir kullanıcı deneyimi? Mobil formlarda genellikle **her alan kendi uyarısını** gösterir; bunu Flutter'da göreceğiz.

3.7–3.8 | İç içe if ve erken çıkış

İç içe if (3.7)

Giriş yapıldı mı? → Premium mu? → Sepet tutarı yeterli mi?
Derinlik arttıkça bloklar zor okunur; koşulları ! ve || ile sadeleştirin.

Erken çıkış / guard clause (3.8)

```
if (kullaniciAdi.isEmpty) { print('...'); return; }
if (yas < 18)             { print('...'); return; }
print('Kayıt başarılı: $kullaniciAdi');
```

Ana fikir: Hatalı durumları önce ele alıp başarılı akışı aşağıda düz bırakmak.

3.9–3.11 | Boş blok, kapsam ve null kontrolü

Kapsam (3.10)

Blok içinde tanımlanan `sonuc` blok dışında kullanılamaz. Değişkeni dışarıda tanımlayın ve **her dalda** değer verin.

Tip yükseltme (3.11)

`String? kuponKodu` için `if (kuponKodu != null)` kontrolünden sonra blok içinde `kuponKodu.length` kullanılabilir.

Boş blok (3.9): Dart'ta boş `{ }` bloğu geçerlidir; Python'daki `pass` gerekmez. Neden boş bırakıldığını bir yorumla belirtin.

VS Code | 3.7–3.11 · Okunur if, kapsam ve null

Dosyalar

`03_12_ic_ice_if.dart` · `03_13_ic_ice_sadelestirme.dart` · `03_14_erken_cikis.dart`
`03_15_bos_blok.dart` · `03_16_tuzak_kapsam.dart` · `03_17_kapsam_dogru.dart`
`03_18_null_kontrol.dart` **Birlikte yazalım**

1. Kargo örneğini önce iç içe, sonra düz `else if` olarak yazın.
2. `return;` ile hatalı girişte erken çıkın; `yas = 20` deneyin.
3. Kapsam hatasındaki `Undefined name` mesajını okuyun; `else` dalını silip yeni hatayı görün.
4. Kupon örneğinde `kuponGirildi = false` deneyin.

Dönüş | 3.7–3.8 · Aynı karar, iki yazım

03_12 iç içe

```
Hoş geldiniz!
Ücretsiz kargo için 30.00 TL daha ekleyin.
```

03_13 sadeleştirilmiş

```
Ücretsiz kargo için 30.00 TL daha ekleyin.
```

Dönüş | 3.7–3.8 · Aynı karar, iki yazım

03_12 iç içe

```
Hoş geldiniz!
Ücretsiz kargo için 30.00 TL daha ekleyin.
```

03_13 sadeleştirilmiş

```
Ücretsiz kargo için 30.00 TL daha ekleyin.
```

03_14 erken çıkış (`yas = 15`)

```
Bu uygulamayı kullanmak için 18 yaşından büyük
olmalısınız.
```

`yas = 20` → `Kayıt başarılı: mert`

Dönüş | 3.7–3.8 · Aynı karar, iki yazım

03_12 iç içe

```
Hoş geldiniz!
Ücretsiz kargo için 30.00 TL daha ekleyin.
```

03_13 sadeleştirilmiş

```
Ücretsiz kargo için 30.00 TL daha ekleyin.
```

03_14 erken çıkış (`yas = 15`)

```
Bu uygulamayı kullanmak için 18 yaşından büyük
olmalısınız.
```

`yas = 20` → `Kayıt başarılı: mert`

Sorgu: Hangi durumlarda aynı sonuç?

Giriş yok	İkisi de: giriş yapmalısınız
Premium	İkisi de: kargo ücretsiz
Tutar ≥ 150	İkisi de: kargo ücretsiz
Diğer	İkisi de: "... TL daha ekleyin"

Fark: Kararlar aynı, **mesajlar** farklı (iç içe sürüm "Hoş geldiniz!" de yazıyor). Davranışı korurken yapıyı değiştirmeye **refactoring** denir.

03_16 derleme hatası

```
Undefined name 'sonuc'.
```

Dönüş | 3.9–3.11 · Kapsam ve null

03_16 derleme hatası

```
Undefined name 'sonuc'.
```

03_17'de `else` dalı silinirse

```
The non-nullable local variable 'sonuc' must be
assigned before it can be used.
```

Dönüş | 3.9–3.11 · Kapsam ve null

03_16 derleme hatası

```
Undefined name 'sonuc'.
```

03_17'de `else` dalı silinirse

```
The non-nullable local variable 'sonuc' must be
assigned before it can be used.
```

03_18 (`kuponGirildi = true`)

```
Kupon uygulandı: OGRENCI10 (9 karakter)
```

`kuponGirildi = false` → `Kupon girilmedi.`

Tartışma

Dart, "puan 50'nin altındaysa `sonuc` ne olacak?" diye soruyor. Bu kontrol, değeri belirsiz değişkenlerin kullanıcıya ulaşmasını engeller.

4 | Üçlü Operatör

Değer seçmek ile işlem yapmak arasındaki fark

Ders notu: Bölüm 4 · Dosyalar: `04_01` – `04_06`

4.1–4.2 | Üçlü operatör: değer seçmenin kısa yolu

```
kosul ? dogruysaDeger : yanlissaDeger
```

if / else (4.1)

`tema2` değişkenine bir dalda "Koyu tema", diğer dalda "Açık tema" atanır: 6 satır.

Üçlü (4.1)

```
var tema = karanlikMod ? 'Koyu tema' :
'Acik tema';
```

Tek satır.

Ayırt edici fikir (4.2)

`if` bir **deyim** (statement); üçlü operatör bir **ifadedir** (expression) ve bir değer üretir. Bu yüzden `print(...)` içine ve metnin içine `${...}` ile yazılabilir.

4.2–4.4 | Üçlüde iki tuzak ve null için kısa yol

Dikkat

- `String tema = if (karanlikMod) ...;` **geçersizdir**: `if` bir değer üretmez (4.2).
- İç içe üçlü mümkündür (4.3): `pil <= 5 ? 'Kritik' : pil <= 20 ? 'Dusuk' : 'Normal'`; ancak okuması zordur.

Null için kısa yol (4.4)

`takmaAd != null ? takmaAd : ad` ≡ `takmaAd ?? ad`

Flutter bağlantısı

`Text(karanlikMod ? 'Koyu' : 'Acik')`: Arayüz ifadelerle kurulur; oraya `if` bloğu yazılamaz.

VS Code | 4.1–4.4 · Üçlü operatör ve null

Dosyalar

`04_01_ternary.dart` · `04_02_ternary_ifade.dart`	**Birlikte yazalım**
`04_03_tuzak_if_deger.dart` · `04_04_ternary_print.dart`	**Öğrenci denesin**
`04_05_ic_ice_ternary.dart` · `04_06_null_ternary.dart`	**Öğrenci denesin**

1. Karanlık mod için `if / else` ve üçlü yazımlarını çalıştırın.
2. Üçlüyü `print(...)` içine ve `${...}` ile metnin içine yerleştirin.
3. İç içe üçlüyü `else if` ile yeniden yazın; `??` ile aynı çıktıyı kısa yoldan elde edin.

Sorgu

İki değerden birini seçerken hangi yapı; iki ayrı işlem yaparken hangi yapı?

Dönüş | 4.1–4.4 · Çıktılar ve alternatif yazımlar

04_02 (`okunmamis = 0`, `sepetAdedi = 3`)

```
Yeni mesaj yok
Sepetinizde 3 ürün var (kargo ücretsiz!).
1 ürün seçildi
```

04_05 (`pil = 15`)

```
Düşük
```

04_06 her iki yazım

```
Ayşe
Ayşe
```

Dönüş | 4.1–4.4 · Çıktılar ve alternatif yazımlar

04_02 (`okunmamis = 0`, `sepetAdedi = 3`)

```
Yeni mesaj yok
Sepetinizde 3 ürün var (kargo ücretsiz!).
1 ürün seçildi
```

04_05 (`pil = 15`)

```
Düşük
```

04_06 her iki yazım

```
Ayşe
Ayşe
```

Alternatif: 04_05'in `else if` hali

```
String durum;
if (pil <= 5) {
    durum = 'Kritik';
  } else if (pil <= 20) {
    durum = 'Düşük';
  } else {
    durum = 'Normal';
  }
```

Karar

İki değer → üçlü. Üç ve daha fazla durum → `else if` veya bir sonraki bölümdeki `switch` ifadesi.

5 | switch

Deyim mi, ifade mi?

Ders notu: Bölüm 5 · Dosyalar: `05_01 - 05_10`

	switch deyimi (5.1–5.3)	**switch ifadesi (5.4–5.6)**
Amaç	İş yapmak	Değer üretmek
Kol	`case 1:`	`1 => sonuc,`
Diğer	`default:`	`_ => sonuc`
Kullanım	Bağımsız yapı	Atamada / değer gereken yerde

Örnekler

Gün numarasına göre mesaj yazmak → deyim.
HTTP durum koduna göre mesaj **seçmek** → ifade.

5.1–5.3 | switch deyimi: bir case seçilir

Örnek yapı

```
switch (gun) {
    case 1:
    print('Pazartesi');
    case 6 || 7:
    print('Hafta sonu');
    default:
    print('Geçersiz gün numarası');
  }
```

- **Dart 3**: içi dolu case sonunda `break;` zorunlu değil (5.1).
- Birden fazla değer: `case 6 || 7:` veya art arda boş case'ler (5.2).
- Metinlerde büyük / küçük harf farkı önemlidir (5.3).

VS Code | 5.1–5.3 · switch deyimi

Dosyalar

`05_01_switch_temel.dart` · `05_02_switch_coklu.dart`
`05_03_switch_bos_case.dart` · `05_04_switch_string.dart` **Birlikte yazalım**

1. 05_01'de gün numarası 3, 6, 9 için sonuçları deneyin.
2. `case 6 || 7:` ile art arda boş case yazımını karşılaştırın.
3. 05_04'te `'kargoda'` yerine `'Kargoda'` yazıp test edin.

Sorgu

Bir case içinde iki `print` satırı kullanmak mümkün mü?

Dönüş | 5.1–5.3 · Çıktılar ve harf duyarlılığı

05_01 *gun → çıktı*

```
3 → Çarşamba
6 → Cumartesi
9 → Geçersiz gün numarası
```

05_02 (gun = 6)

```
Hafta sonu: Bildirimler sessize alındı.
```

Dönüş | 5.1–5.3 · Çıktılar ve harf duyarlılığı

05_01 *gun → çıktı*

```
3 → Çarşamba
6 → Cumartesi
9 → Geçersiz gün numarası
```

05_02 (gun = 6)

```
Hafta sonu: Bildirimler sessize alındı.
```

05_04 (’kargoda’): bir case, iki satır

```
Siparişiniz yolda.
Tahmini teslimat: 2 gün.
```

’Kargoda’ → Bilinmeyen durum: Kargoda

05_01 *gun → çıktı*

```
3 → Çarşamba
6 → Cumartesi
9 → Geçersiz gün numarası
```

05_02 (gun = 6)

```
Hafta sonu: Bildirimler sessize alındı.
```

05_04 ('kargoda'): bir case, iki satır

```
Siparişiniz yolda.
Tahmini teslimat: 2 gün.
```

'Kargoda' → Bilinmeyen durum: Kargoda

Alternatif: Kullanıcı girdisinde güvenli yol

```
switch (siparisDurumu.toLowerCase()) { ... }
```

5.4 | switch ifadesi: değer üreten desen

HTTP yanıt kodu → mesaj

```
String mesaj = switch (durumKodu) {
    200 => 'İşlem başarılı.',
    401 || 403 => 'Yetkiniz yok, lütfen tekrar giriş yapın.',
    404 => 'Aradığınız içerik bulunamadı.',
    500 => 'Sunucu hatası, daha sonra tekrar deneyin.',
    _ => 'Beklenmeyen durum: $durumKodu',
  };
```

Okuma kuralı: => sonuç, , kol ayracı, _ varsayılan durum, en sonda };

VS Code | 5.4 · switch ifadesi ve eksiksizlik

Dosyalar

`05_05_switch_ifadesi.dart` · `05_06_tuzak_eksik_durum.dart` · `05_07_switch_bool.dart`
Birlikte yazalım

1. `200, 403, 404, 500, 302` için mesajları sınayın.
2. `_ => ...` kolunu kaldırıp derleme hatasını okuyun.
3. `bool` üzerinde hem `true` hem `false` kolu varken `_` gerekip gerekmediğini düşünün.

Sorgu

Switch ifadesi neden her olası girdi için bir değer üretebilmelidir?

05_05 *durumKodu → mesaj*

```
200 → İşlem başarılı.
403 → Yetkiniz yok, lütfen tekrar giriş yapın.
404 → Aradığınız içerik bulunamadı.
500 → Sunucu hatası, daha sonra tekrar deneyin.
302 → Beklenmeyen durum: 302
```

05_05 *durumKodu → mesaj*

```
200 → İşlem başarılı.
403 → Yetkiniz yok, lütfen tekrar giriş yapın.
404 → Aradığınız içerik bulunamadı.
500 → Sunucu hatası, daha sonra tekrar deneyin.
302 → Beklenmeyen durum: 302
```

05_06 _ kolu yokken

```
The type 'int' isn't exhaustively matched by the
switch cases since it doesn't match 'int()'.
```

05_07 (`sarjda = true`)

```
Şarj oluyor (%64)
```

`bool`'un yalnızca iki değeri var; ikisi de yazıldığı için _ gerekmez.

5.5–5.6 | İlişkisel desenler ve when

Harf notu (5.5)

```
String harf = switch (puan) {
    < 0 || > 100 => 'Geçersiz puan',
    >= 90 => 'AA',
    >= 85 => 'BA',
    >= 80 => 'BB',
    >= 75 => 'CB',
    // ... CC, DC, DD
    _ => 'FF',
  };
```

Sıra yine önemli

105 puanı ilk yakalayan kol `>= 90` olmasın: geçersiz aralığı **en başa** koyun.

Aralık (5.5) ve koruma (5.6)

`>= 600 && < 840 => ...`

`_ when premium => ...`

`when`: "Desen eşleşse bile şu koşul da doğru mu?"

VS Code | 5.5–5.6 · Aralık, ekran genişliği ve when

Dosyalar

`05_08_iliskisel_desen.dart` · `05_09_ekran_genisligi.dart` · `05_10_when.dart` **Birlikte yazalım**

1. 78 puan için harf notunu; 105 için geçersiz sonucu bulun.
2. Ekran genişliği 500, 700, 900 olduğunda düzeni yorumlayın.
3. `premium = false` yapıp kargo sonucunu karşılaştırın.

Sorgu

`if` yerine `switch` ne zaman daha okunur? `when` hangi ek soruyu sorar?

05_08 *puan → çıktı*

```
78 → Puan: 78 → Harf notu: CB
105 → Puan: 105 → Harf notu: Geçersiz puan
```

05_08 *puan → çıktı*

```
78 → Puan: 78 → Harf notu: CB
105 → Puan: 105 → Harf notu: Geçersiz puan
```

05_09 *genislik → çıktı*

```
500 → 500.0 dp → Kompakt: telefon (tek sütun)
700 → 700.0 dp → Orta: tablet veya katlanabilir (iki sütun)
900 → 900.0 dp → Geniş: yatay tablet veya masaüstü (yan menü)
```

05_08 *puan → çıktı*

```
78 → Puan: 78 → Harf notu: CB
105 → Puan: 105 → Harf notu: Geçersiz puan
```

05_09 *genislik → çıktı*

```
500 → 500.0 dp → Kompakt: telefon (tek sütun)
700 → 700.0 dp → Orta: tablet veya katlanabilir (iki sütun)
900 → 900.0 dp → Geniş: yatay tablet veya masaüstü (yan menü)
```

05_10 *premium, sepetTutari → çıktı*

```
true,  120 → Kargo: Ücretsiz (Premium üye)
false, 120 → Kargo: 39,90 TL
false, 200 → Kargo: Ücretsiz
```

5.7 | if mi, switch mi? Hangisini seçersiniz?

Durum	Yapı
İnternet varsa yükle, yoksa uyar	`if / else`
Kullanıcının giriş, sepet ve adres durumunu doğrula	`if / else if`
Koyu / açık tema metni seç	`? :`
HTTP koduna göre mesaj üret	`switch` ifadesi
Menü seçimine göre işlem çalıştır	`switch` deyimi

Kural: Farklı değişkenlere bakan birleşik koşullar → `if`. **Tek bir değişkenin** birçok değeri veya aralığı → `switch`.

6 | Operatörler – Bölüm 1

Aritmetik, metin, atama, artırma ve karşılaştırma

Ders notu: Bölüm 6 · Dosyalar: `06_01` – `06_10`

6.1 | Aritmetik operatörler

İşlem	Dart	`a = 17, b = 5`
Toplama / çıkarma	+, −	22, 12
Çarpma	*	85
Bölme (`double`)	/	3.4
Tam sayı bölmesi	~/	3
Kalan	%	2

Dil farkı

Python'da tam sayı bölmesi `//`; Dart'ta `~/`. Dart'ta `//` yorum başlatır.

VS Code | 6.1 · Aritmetik, tip ve sıfıra bölme

Dosyalar

`06_01_aritmetik.dart` · `06_02_sonuc_tipleri.dart`
`06_03_bolme_kalan.dart` · `06_04_sifira_bolme.dart` **Birlikte yazalım**

1. `17 / 5`, `17 ~/ 5`, `17 % 5` çıktısını kıyaslayın.
2. `runtimeType` ile sonuç tiplerine bakın.
3. 227 saniyeyi dakika + saniyeye dönüştürün; 23 ürünün kaç sayfa tuttuğunu bulun.
4. `10 / 0` ile yorumdaki `10 ~/ 0` satırını karşılaştırın.

Sorgu

`10 / 4` ve `10 ~/ 4` neden farklı türde sonuç verir?

06_02

```
148.5
2.5
2
double
int
```

06_03

```
Şarkı süresi: 3 dk 47 sn
7 tek sayıdır.
23 ürün, sayfa başına 10 → 3 sayfa
```

06_04

```
Infinity
-Infinity
NaN
```

06_02

```
148.5
2.5
2
double
int
```

06_03

```
Şarkı süresi: 3 dk 47 sn
7 tek sayıdır.
23 ürün, sayfa başına 10 → 3 sayfa
```

06_04

```
Infinity
-Infinity
NaN
```

Tartışma

`/` her zaman `double`, `~/` her zaman `int` üretir.
Sayfalama: `(23 + 10 - 1) ~/ 10 = 3`. Bölünene `sayfaBasina - 1` eklemek yukarı yuvarlar.
`double` bölmede sıfır çökme yapmaz (`Infinity`, `NaN`); `~/ 0` çalışma zamanı hatasıdır.

6.2–6.3 | Metin operatörleri ve bileşik atama

Metinler (6.2)

`ad + ' ' + soyad` birleştirir.
`'-' * 20` metni tekrarlar.
`'Yas: ' + yas` **hata**; doğrusu: `'Yaş: $yas'`

Bileşik atama (6.3)

`a += 3`	`a = a + 3`	biriktirme
`a -= 3`	`a = a - 3`	azaltma
`a *= 3`	`a = a * 3`	ölçekleme
`a ~/= 3`	`a = a ~/ 3`	tam bölme
`a %= 3`	`a = a % 3`	kalanı ata

Dikkat

`int` bir değişkende `/=` kullanılamaz: sonuç `double` olur.

6.4–6.5 | Artırma, azaltma ve karşılaştırma

Soru (6.4): `a = 5` ve `c = 5` iken

`b = a++;` ve `d = ++c;` sonrasında `a, b, c, d` kaçtır?

Sonek: önce kullan, sonra artır. Önek: önce artır, sonra kullan.

Karşılaştırma (6.5)

`10 == 10.0` → `true`

Metinlerde `==` içeriği karşılaştırır.

Metinlerde `<` **yoktur**; sıralama için `compareTo`.

VS Code | 6.2–6.5 · Metin, atama ve karşılaştırma

Dosyalar

`06_05_string_operatorleri.dart` · `06_06_tuzak_string_sayi.dart`
`06_07_atama.dart` · `06_08_artirma_azaltma.dart`
`06_09_karsilastirma.dart` · `06_10_tuzak_metin_karsilastirma.dart` **Birlikte yazalım**

1. Metin + sayı hatasını interpolation ile giderin.
2. +=, %= işlemlerini adım adım izleyin.
3. a++ ile ++a farkını çıktılarda bulun.
4. Metinlerde < neden kullanılamıyor; `compareTo` deneyin.

06_07 atama zinciri

```
Başlangıç : 100
+= 20     : 120
-= 30     : 90
*= 2      : 180
~/= 3     : 60
%= 7      : 4
bakiye /= 4 → 25.0
```

06_08

```
6
5
a = 6, b = 5
c = 6, d = 6
```

Dönüş | 6.2–6.5 · Çıktılar ve hatalar

06_07 atama zinciri

```
Başlangıç : 100
+= 20     : 120
-= 30     : 90
*= 2      : 180
~/= 3     : 60
%= 7      : 4
bakiye /= 4 → 25.0
```

06_08

```
6
5
a = 6, b = 5
c = 6, d = 6
```

06_05

```
Cep Market
Cep Market
--------------------
Değerlendirme: ****
```

06_06 ve **06_10** derleme hataları

```
The argument type 'int' can't be assigned to the parameter
type 'String'.
The operator '<' isn't defined for the type 'String'.
```

06_09: `true true true false true true`
60 % 7 = 4 çünkü $60 = 7 \times 8 + 4$.

7 | Operatörler – Bölüm 2

Mantık, kısa devre, tip, null, bit ve öncelik

Ders notu: Bölüm 7 · Dosyalar: `07_01` – `07_06`

7.1 | Mantıksal operatörlerin doğruluk tablosu

A	B	`A && B`	`A || B`	`!A`
true	true	true	true	false
true	false	false	true	false
false	true	false	true	true
false	false	false	false	true

Ödeme butonu

`girisYapti && !sepetBos && adresVar` ancak tüm şartlar sağlanırsa doğrudur.

7.2 | Kısa devre (short-circuit) değerlendirme

Sağ taraf ne zaman değerlendirilmez?

- `false && ...` → sonuç zaten `false`.
- `true || ...` → sonuç zaten `true`.

Null güvenliği

```
kullaniciAdi != null && kullaniciAdi.length > 3
```

Önce `null` kontrolü, sonra özellik erişimi. **Sıra değiştirilmez.**

7.3–7.5 | Tip testi, null ve bit operatörleri

Tip (7.3)

`x is int`
`x is! String`
`x as String`

as yanlışsa çalışma zamanı hatası.

Null (7.4)

`x ?? y`: null ise y
`x ??= y`: null ise ata
`x?.alan`: null ise null

Bit (7.5, bilgi)

& VE, | VEYA
^ XOR
«, » kaydırma

Örnek: 6 (110_2) ve 3 (011_2) için bit VE sonucu 2'dir (010_2).

VS Code | 7.1–7.5 · Mantık, tip, null ve bit

Dosyalar

`07_01_mantiksal.dart` · `07_02_kisa_devre.dart` · `07_03_tip_test.dart`
`07_04_null_operatorleri.dart` · `07_05_bit_operatorleri.dart` **Birlikte yazalım**

1. `sepetBos = true` yapınca ödeme butonunun durumunu izleyin.
2. Kısa devre örneğinde `kullaniciVar = true` deneyin.
3. `is`, `is!`, `as` ile tür kontrolü ve dönüşümünü karşılaştırın.
4. 6 ile 3 için bit VE, VEYA, XOR sonuçlarını bulun.

Sorgu

Mantıksal `&&` ile bit düzeyi `&` aynı amaçla mı kullanılır?

Dönüş | 7.1–7.5 · Çıktılar

07_01

```
Ödeme butonu aktif mi? true
İndirim uygulanacak mı? true
```

07_02 (`kullaniciVar = false`)

```
Kullanıcı adı yok veya çok kısa.
Lütfen bir kullanıcı adı girin.
```

07_03

```
Tam sayı geldi: 43
Gelen veri bir metin değil.
MERHABA
```

07_04

```
varsayilan_avatar.png
null
varsayilan_avatar.png
```

07_05: `2 7 5 12 3 110`

07_01

```
Ödeme butonu aktif mi? true
İndirim uygulanacak mı? true
```

07_02 (`kullaniciVar = false`)

```
Kullanıcı adı yok veya çok kısa.
Lütfen bir kullanıcı adı girin.
```

07_03

```
Tam sayı geldi: 43
Gelen veri bir metin değil.
MERHABA
```

07_04

```
varsayilan_avatar.png
null
varsayilan_avatar.png
```

07_05: `2 7 5 12 3 110`

Tartışma

`&&` iki `bool` değeri birleştirir ve kısa devre yapar. `&` sayıların **bitleri** üzerinde çalışır. Koşullarda her zaman `&&` ve `||`.

7.6 | Operatör önceliği: önemli katmanlar

1. `( )` ile açık gruplama
2. `* / ~/ %` → `+ -`
3. Karşılaştırma → eşitlik
4. `&&` → `||`
5. `??` → `? :` → atama

Önce tahmin edin (07_06_oncelik.dart)

```
2 + 3 * 4    (2 + 3) * 4    10 - 4 - 3    -2 * -3    a || b && c    (a || b) && c
```

Kural: Emin değilseniz parantez kullanın.

07_06

07_06

```
14
20
3
6
true
false
```

Neden?

`2 + 3 * 4` → önce çarpma: `2 + 12 = 14`
`10 - 4 - 3` → soldan sağa: `(10 - 4) - 3 = 3`
`a || b && c` → `&&` önce: `true || false = true`
`(a || b) && c` → `true && false = false`

Okunabilirlik: `(a && b) || c` yazmak, sonucu değiştirmese bile okuyanın işini kolaylaştırır.

8 | for Döngüsü

Tekrar sayısı belli olduğunda

Ders notu: Bölüm 8 · Dosyalar: `08_01` – `08_10`

8.1 | Neden döngü kullanırız?

Problem

Aynı bildirimi 3 kez yazdırmak için üç `print` satırı yazabilirsiniz. Peki 300 kez?

`for` `while` `do-while`

- Tekrar sayısı belliyse: `for` (Bölüm 8)
- Ne kadar süreceği koşula bağlıysa: `while` (Bölüm 9)
- Gövde mutlaka en az bir kez çalışacaksa: `do-while` (Bölüm 10)

8.2 | for: başlangıç, koşul, artış

```
for (int i = 0; i < 4; i++)
```

Parça	Ne zaman çalışır?	Görev
`int i = 0`	Bir kez	Sayacı başlat
`i < 4`	Her tur öncesi	Devam edilsin mi?
`i++`	Her turun sonunda	Sayacı artır

Kaç tur?

`for (int i = 0; i < n; i++)` tam **n** tur döner; sayaç değerleri 0, 1, . . . , n-1.

VS Code | 8.1–8.2 · for anatomisi ve izleme

Dosyalar

`08_01_neden_dongu.dart` · `08_02_for_yapisi.dart` **Birlikte yazalım**

1. İlk örneği döngüsüz ve döngülü yazımla karşılaştırın.
2. `i = 0, 1, 2, 3, 4` için koşulun değerini tahtada izleyin.
3. `i < 4` yerine `i <= 4` yazıldığında ne değiştiğini görün.

Sorgu

Döngü 4 kez mi, 5 kez mi çalıştı? “Bir eksik / bir fazla” (off-by-one) hatası nereden çıkar?

08_02

```
i = 0
i = 1
i = 2
i = 3
Döngü bitti.
```

08_02

```
i = 0
i = 1
i = 2
i = 3
Döngü bitti.
```

Adım	`i`	`i < 4`	Ne olur?
1	0	true	yazdır, `i++` → 1
2	1	true	yazdır, `i++` → 2
3	2	true	yazdır, `i++` → 3
4	3	true	yazdır, `i++` → 4
Kontrol	4	false	döngü biter

08_02

```
i = 0
i = 1
i = 2
i = 3
Döngü bitti.
```

Adım	`i`	`i < 4`	Ne olur?
1	0	true	yazdır, `i++` → 1
2	1	true	yazdır, `i++` → 2
3	2	true	yazdır, `i++` → 3
4	3	true	yazdır, `i++` → 4
Kontrol	4	false	döngü biter

`i <= 4` olsaydı

5 tur döner (0–4). Listelerle çalışırken bu, olmayan bir elemana erişmeye yol açar.

8.3 | for için temel kalıplar

İhtiyaç	Örnek fikir	Dosya
Geri sayım	`saniye--` ile 5'ten 1'e	08_03
Farklı adım	`yuzde += 25`	08_04
Toplam / biriktirme	`toplamAdim += gunlukAdim`	08_05
İlerleme göstergesi	`'#' * adim`	08_06

Kapsam

Biriktirme değişkenini döngünün **dışında** tanımlayın. `for (int i ...)` ile oluşturulan `i` ise döngü dışında erişilemez (8.6).

VS Code | 8.3 · for kalıplarını birlikte yapalım

Dosyalar

`08_03_geri_sayim.dart` · `08_04_adim.dart`
`08_05_toplam.dart` · `08_06_ilerleme_cubugu.dart` **Öğrenci yazsın**

1. Doğrulama kodu için 5'ten 1'e geri sayın; yüklemeyi %25'er artırın.
2. 7 günlük adım sayısını biriktirip toplam ve ortalamayı bulun.
3. Metin tekrarıyla `[####---]` biçiminde ilerleme çubuğu çizin.

Sorgu

`toplamAdim = 0` satırını döngünün içine taşımak neyi bozar?

Dönüş | 8.3 · Kalıpların çıktıları

08_04

```
Yükleniyor... %0
Yükleniyor... %25
Yükleniyor... %50
Yükleniyor... %75
Yükleniyor... %100
```

08_05 (son iki satır)

```
Haftalık toplam: 49000 adım
Günlük ortalama: 7000 adım
```

08_06

```
[----------] %0
[##--------] %20
[####------] %40
[######----] %60
[########--] %80
[##########] %100
```

08_04

```
Yükleniyor... %0
Yükleniyor... %25
Yükleniyor... %50
Yükleniyor... %75
Yükleniyor... %100
```

08_05 (son iki satır)

```
Haftalık toplam: 49000 adım
Günlük ortalama: 7000 adım
```

08_06

```
[----------] %0
[##--------] %20
[####------] %40
[######----] %60
[########--] %80
[##########] %100
```

toplamAdim döngünün içinde olsaydı

Her turda sıfırlanırdı; ayrıca döngüden sonra erişilemezdi ve son `print` satırları derlenmezdi.

8.4–8.5 | İç içe for (ızgara) ve for-in

3 satır, 4 sütun (8.4)

0	1	2	3
4	5	6	7
8	9	10	11

Dış döngü satırlar, iç döngü sütunlar: $3 \times 4 = 12$ tur.
İndeks: `satir * sutunSayisi + sutun`

for-in (8.5)

`for (var b in bildirimler)`

"Listedeki **her** eleman için..."

Sayaç, koşul, artış yok; indeks hatası riski yok.

Listeleri 3. haftada ayrıntılı göreceğiz.

VS Code | 8.4–8.6 · Izgara, for-in ve kapsam

Dosyalar

`08_07_ic_ice_for.dart` · `08_08_for_in.dart`
`08_09_for_in_metin.dart` · `08_10_tuzak_dongu_kapsam.dart` **Birlikte yazalım**

1. 3x4 ızgarayı yazdırın; `satir = 1, sutun = 2` hücresinin indeksini bulun.
2. Bildirimler listesini `for-in` ile dolaşın.
3. `'Flutter2026'` içindeki rakamları sayın; `i` kapsam hatasını inceleyin.

Sorgu

`for-in` ile indeksli `for` arasında seçim yaparken ölçütümüz ne?

08_07

```
[ 0] [ 1] [ 2] [ 3]
[ 4] [ 5] [ 6] [ 7]
[ 8] [ 9] [10] [11]
```

`satir = 1, sutun = 2` $\rightarrow$ `1 * 4 + 2 = 6`

08_09

```
"Flutter2026" içinde 4 rakam var.
```

08_10 derleme hatası

```
Undefined name 'i'.
```

08_07

```
[ 0] [ 1] [ 2] [ 3]
[ 4] [ 5] [ 6] [ 7]
[ 8] [ 9] [10] [11]
```

`satir = 1, sutun = 2` → `1 * 4 + 2 = 6`

08_09

```
"Flutter2026" içinde 4 rakam var.
```

08_10 derleme hatası

```
Undefined name 'i'.
```

Seçim ölçütü

Yalnızca **elemanlar** gerekiyorsa `for-in`. **İndeks** de gerekiyorsa (sıra numarası yazdırmak, iki listeyi eşleştirmek) indeksli `for`.

Flutter: `GridView` her hücre için bir indeks verir; satır = `indeks ~/ 4`, sütun = `indeks % 4`.

9 | while Döngüsü

Tur sayısını baştan bilmediğimizde

Ders notu: Bölüm 9 · Dosyalar: `09_01` – `09_04`

9.1–9.4 | while: önce koşul, sonra gövde

Mantık (9.1)

`while (kosul)`: Koşulu **önce** değerlendir; doğruysa gövdeyi çalıştır; yeniden kontrol et.

Örnekler

9.1 Birikim hedefe kaç ayda ulaşır? · **9.2** Sunucuya bağlanana kadar yeniden dene · **9.3** Sayının basamaklarını ayır

Sonsuz döngü riski (9.4)

Koşulu `false` yapacak bir değişim olmalı; beklenmeyen senaryolara **üst sınır** koyun (`deneme < maksimumDeneme`).

VS Code | 9.1–9.4 · while ve yeniden deneme

Dosyalar

`09_01_while_temel.dart` · `09_02_yeniden_deneme.dart`
`09_03_basamaklar.dart` · `09_04_tuzak_sonsuz_dongu.dart` **Birlikte yazalım**

1. `aylikTasarruf = 1000` yapın; kaç ay sürer? Önce tahmin.
2. Simülasyondaki 3'ü 8 yapıp güvenlik sınırını gözlemleyin.
3. Basamakları `% 10` ve `~/ 10` ile ayrıştırın.
4. Sonsuz döngüyü inceleyin; çalıştırırsanız durdurmak için `Ctrl + C`.

Sorgu

Bir `while` döngüsünün sonlanacağını nasıl gerekçelendiririz?

Dönüş | 9.1–9.4 · Çıktılar ve gerekçeler

09_01 (son iki satır)

```
7. ay sonunda birikim: 5250.00 TL
Hedefe 7 ayda ulaşıldı.
```

`aylikTasarruf = 1000` → `Hedefe 5 ayda ulaşıldı.`

09_02

```
Sunucuya bağlanılıyor... (deneme 1/5)
Sunucuya bağlanılıyor... (deneme 2/5)
Sunucuya bağlanılıyor... (deneme 3/5)
Bağlantı kuruldu.
```

09_03

```
Kod: 482915
Basamak sayısı: 6
Basamak toplamı: 29
```

Dönüş | 9.1–9.4 · Çıktılar ve gerekçeler

09_01 (son iki satır)

```
7. ay sonunda birikim: 5250.00 TL
Hedefe 7 ayda ulaşıldı.
```

`aylikTasarruf = 1000` → `Hedefe 5 ayda ulaşıldı.`

09_02

```
Sunucuya bağlanılıyor... (deneme 1/5)
Sunucuya bağlanılıyor... (deneme 2/5)
Sunucuya bağlanılıyor... (deneme 3/5)
Bağlantı kuruldu.
```

09_03

```
Kod: 482915
Basamak sayısı: 6
Basamak toplamı: 29
```

Sonlanma gerekçesi

09_01: birikim her turda **artıyor**. 09_02: deneme her turda **artıyor** ve bir sınırı var. 09_03: sayı her turda **10'a bölünerek** küçülüyor. 09_04: sayac hiç değişmiyor → sonsuz.

10 | do-while Döngüsü

Gövde en az bir kez çalışmalıysa

Ders notu: Bölüm 10 · Dosyalar: `10_01` – `10_03`

10.1–10.2 | while ve do-while karşılaştırması

	`while`	`do-while`
Koşul	Gövdeden önce	Gövdeden sonra
En az tur	0	1
Kullanım	Önkoşul, bekleme	Menü, ilk girdi alma
Noktalı virgül	`while (...) {...}`	`do {...} while (...);`

Tahmin (10.2)

Koşul başlangıçta `false` ise iki yapı kaçar kez çalışır?

VS Code | 10.1–10.3 · do-while ve PIN sınırı

Dosyalar

`10_01_do_while.dart` · `10_02_while_vs_do_while.dart` · `10_03_pin_simulasyon.dart`
Birlikte yazalım

1. Tanıtım ekranları için en az bir kez çalışan döngüyü yazın.
2. Aynı `false` koşulda iki döngünün çıktılarını karşılaştırın.
3. PIN örneğinde son denemeyi `'0000'` yapın.

Sorgu

`do-while` sonundaki `;` neden unutulmamalı?

Dönüş | 10.1–10.3 · Çıktılar

10_01

```
Tanıtım ekranı 1/3
Tanıtım ekranı 2/3
Tanıtım ekranı 3/3
Ana sayfaya geçiliyor...
```

10_02: yalnızca bir satır

```
do-while gövdesi çalıştı
```

10_03

```
Deneme 1: 1111
Deneme 2: 1234
Deneme 3: 2580
Kilit açıldı.
```

10_01

```
Tanıtım ekranı 1/3
Tanıtım ekranı 2/3
Tanıtım ekranı 3/3
Ana sayfaya geçiliyor...
```

10_02: yalnızca bir satır

```
do-while gövdesi çalıştı
```

10_03

```
Deneme 1: 1111
Deneme 2: 1234
Deneme 3: 2580
Kilit açıldı.
```

Son deneme `'0000'` olursa:

```
Deneme 3: 0000
Çok fazla hatalı deneme!
```

Noktalı virgül: `do-while` bir ifade gibi biter; `while (kosul);` zorunludur. Normal `while`'da ise konmaz.

11 | break, continue ve Etiketler

Döngü akışını yönetmek

Ders notu: Bölüm 11 · Dosyalar: `11_01 - 11_09`

11.1–11.3 | break ve continue aynı şey değil

break (11.1)

En yakın **döngüyü tamamen bitir**.

Örn. aranan ilk siparişi bulduğunda aramayı bırak.

continue (11.2)

Bu turun **kalanını atla**; bir sonraki tura geç.

Örn. sessiz saatlerde bildirim göndermeyi atla.

while (true) + break (11.3)

Çıkış noktası gövdenin **ortasında** belli oluyorsa sonsuz döngü kurulur ve çıkış `break` ile yapılır. Döngüde mutlaka **ulaşılabilir** bir `break` olmalıdır.

VS Code | 11.1–11.3 · break / continue / while(true)

Dosyalar

`11_01_break.dart` · `11_02_continue.dart` · `11_03_while_true.dart` **Birlikte yazalım**

1. Hem 7'ye hem 11'e bölünen ilk sipariş numarasını `break` ile bulun.
2. 23:00–07:00 sessiz saatlerini `continue` ile atlayın.
3. İndirme simülasyonunda `break`'in nerede olduğunu gösterin.

Sorgu

Bir turu atlamak ile döngüyü sonlandırmak çıktıyı nasıl değiştirir?

Dönüş | 11.1–11.3 · Çıktılar

11_01

```
İkisini birden kazanan ilk sipariş: 77
Arama bitti.
```

11_02

```
00:00 → sessiz saat, atlandı
03:00 → sessiz saat, atlandı
06:00 → sessiz saat, atlandı
09:00 → su içme hatırlatması gönderildi
12:00 → su içme hatırlatması gönderildi
15:00 → su içme hatırlatması gönderildi
18:00 → su içme hatırlatması gönderildi
21:00 → su içme hatırlatması gönderildi
```

11_03

```
İndiriliyor... %30
İndiriliyor... %60
İndiriliyor... %90
İndirme tamamlandı (%100).
```

Dönüş | 11.1–11.3 · Çıktılar

11_01

```
İkisini birden kazanan ilk sipariş: 77
Arama bitti.
```

11_02

```
00:00 → sessiz saat, atlandı
03:00 → sessiz saat, atlandı
06:00 → sessiz saat, atlandı
09:00 → su içme hatırlatması gönderildi
12:00 → su içme hatırlatması gönderildi
15:00 → su içme hatırlatması gönderildi
18:00 → su içme hatırlatması gönderildi
21:00 → su içme hatırlatması gönderildi
```

11_03

```
İndiriliyor... %30
İndiriliyor... %60
İndiriliyor... %90
İndirme tamamlandı (%100).
```

Tartışma

11_01'de `break` olmasaydı döngü 1000'e kadar dönerdi (ortak katlar 154, 231, ... da yazılırdı). 11_02'de `continue` yerine `break` olsaydı saat 00:00'da her şey biterdi.

11.4–11.6 | Etiketler: iç içe döngüden dışarı çıkış

Normal break (11.4)

İç içe döngüde `break;` yalnızca **en yakın döngüyü** bitirir.

Etiketli break (11.5)

```
arama:
for (...) {
    for (...) {
        if (indeks == arananIndeks) break arama;
    }
}
```

Etiketli continue (11.6): `continue gunler;` ile bir sonraki **dış döngü turuna** geçilir.

VS Code | 11.4–11.6 · Etiketli akış

Dosyalar

`11_04_ic_ice_break.dart` · `11_05_label_break.dart` · `11_06_label_continue.dart`
Birlikte yazalım

1. 11_04'te `break;`'in yalnızca iç döngüyü bitirdiğini gözleyin.
2. 11_05'te uygulama simgesini bulunca iki döngüden çıkın; sonra `break arama;` yerine `break;` yazın.
3. 11_06'da `continue gunler;` ile günün kalan saatlerini atlayın.

Sorgu

Etiket, komutun hangi döngüye uygulanacağını nasıl belirliyor?

Dönüş | 11.4–11.6 · Etiketli ve etiketsiz

11_04

```
i = 1, j = 1
i = 2, j = 1
i = 3, j = 1
```

11_05 (son üç satır)

```
Kontrol: satır 1, sütun 2 (indeks 6)
Bulundu! Simge 1. satır, 2. sütunda.
Arama tamamlandı.
```

11_06 (4.–7. satırlar)

```
Gün 1, 12:00 → hatırlatma
Gün 2, 9:00 → hatırlatma
Gün 2: Hatırlatmalar bugün için
kapatıldı.
Gün 3, 9:00 → hatırlatma
```

Dönüş | 11.4–11.6 · Etiketli ve etiketsiz

11_04

```
i = 1, j = 1
i = 2, j = 1
i = 3, j = 1
```

11_05 (son üç satır)

```
Kontrol: satır 1, sütun 2 (indeks 6)
Bulundu! Simge 1. satır, 2. sütunda.
Arama tamamlandı.
```

11_06 (4.–7. satırlar)

```
Gün 1, 12:00 → hatırlatma
Gün 2, 9:00 → hatırlatma
Gün 2: Hatırlatmalar bugün için
kapatıldı.
Gün 3, 9:00 → hatırlatma
```

`break arama;` yerine `break;`

İç döngü biter ama dış döngü 2. satırı da tarar: 7 yerine **11** kontrol satırı yazılır. Aranan bulunduğu halde gereksiz iş yapılır.

11_06

Etiketsiz `continue;` yalnızca 10:00'u atlardı; 11:00 ve 12:00 hatırlatmaları yine gönderilirdi.

11.7 | Önemli tuzak: switch içindeki break

Düşünelim

`while (true)` içindeki `switch (secim)` yapısında `case '0': break;` yazıldı. Neden menü tekrar görünüyor?

11.7 | Önemli tuzak: switch içindeki break

Düşünelim

`while (true)` içindeki `switch (secim)` yapısında `case '0': break;` yazıldı. Neden menü tekrar görünüyor?

Çünkü...

`break` en yakın **switch** yapısından çıkar; dıştaki `while` devam eder.

Çözüm A: `menu: while (true)` ve `break menu;`
Çözüm B: `bool devam = true;` bayrağıyla döngüyü sonlandırmak.

VS Code | 11.7 · Switch + döngü çıkışını düzeltelim

Dosyalar

`11_07_tuzak_switch_break.dart` · `11_08_label_menu.dart` · `11_09_bayrak_menu.dart`
Birlikte yazalım

1. Önce hatalı menünün davranışını gözlemleyin (güvenlik sınırı sayesinde durur).
2. Aynı menüyü etiketli `break menu;` ile sonlandırın.
3. Bayrak değişkenli çözümün akışını karşılaştırın.

Sorgu

İki çözümden hangisini daha okunur buldunuz? Neden?

11_07 `break;` (hatalı)

```
Profil gösteriliyor
Ayarlar gösteriliyor
Çıkış seçildi
Çıkış seçildi
Çıkış seçildi
(Güvenlik sınırı: döngü zorla durduruldu)
```

11_07 `break;` (hatalı)

```
Profil gösteriliyor
Ayarlar gösteriliyor
Çıkış seçildi
Çıkış seçildi
Çıkış seçildi
(Güvenlik sınırı: döngü zorla durduruldu)
```

11_08 `break menu;` ve **11_09** bayrak

```
Profil gösteriliyor
Ayarlar gösteriliyor
Çıkış seçildi. Güle güle!
Program sonlandı.
```

11_07 `break;` (hatalı)

```
Profil gösteriliyor
Ayarlar gösteriliyor
Çıkış seçildi
Çıkış seçildi
Çıkış seçildi
(Güvenlik sınırı: döngü zorla durduruldu)
```

11_08 `break menu;` ve **11_09** bayrak

```
Profil gösteriliyor
Ayarlar gösteriliyor
Çıkış seçildi. Güle güle!
Program sonlandı.
```

Hangisi?

İkisi de doğru. Etiket daha kısa; bayrak, etiketleri yabancı bulanlar için daha tanıdık. Ekipte **tutarlı** olan seçilir.

12 | Kullanıcıdan Veri Alma

Terminal, nullable metin, güvenli sayıya dönüşüm

Ders notu: Bölüm 12 · Dosyalar: `12_01` – `12_10`

12.1–12.2 | Klavye girişi metin olarak başlar

dart:io ile konsol

```dart
import 'dart:io';

stdout.write('Adınız: ');
String? ad = stdin.readLineSync();
```

- `stdout.write`: alt satıra geçmeden sorar (12.1).
- `readLineSync()`: Enter'a kadar bekler; türü `String?` (12.1).
- `null` mümkündür: girdi akışı kapanabilir (`Ctrl + D` / `Ctrl + Z`).
- Güvenli kalıp (12.2): `(stdin.readLineSync() ?? "").trim()`

VS Code | 12.1–12.2 · İlk girdi ve boşluk temizleme

Terminalden çalıştırın

```
dart run 12_01_ilk_girdi.dart
```

Dosyalar

`12_01_ilk_girdi.dart` · `12_02_null_ve_trim.dart` **Birlikte yazalım**

1. Adınızı girin; sonra ad yerine `Ctrl + D` (Windows: `Ctrl + Z` ve Enter) deneyin.
2. Girdinin başına ve sonuna boşluk koyarak `trim()` etkisini görün.
3. Yalnızca boşluk yazıp Enter'a basın.

Dönüş | 12.1–12.2 · Örnek oturumlar

12_01

```
Adınız: Ayşe
Merhaba, Ayşe!
```

`Ctrl + D` ile:

```
Adınız: Merhaba, null!
```

12_02 (`" mert "` girildi)

```
Kullanıcı adınız:    mert
Hoş geldin, mert! (4 karakter)
```

Yalnızca boşluk girildi:

```
Kullanıcı adı boş bırakılamaz.
```

Dönüş | 12.1–12.2 · Örnek oturumlar

12_01

```
Adınız: Ayşe
Merhaba, Ayşe!
```

`Ctrl + D` ile:

```
Adınız: Merhaba, null!
```

12_02 (`" mert "` girildi)

```
Kullanıcı adınız:    mert
Hoş geldin, mert! (4 karakter)
```

Yalnızca boşluk girildi:

```
Kullanıcı adı boş bırakılamaz.
```

Tartışma

`??` `"` sayesinde `girdi`'nin tipi `String` oldu; `!` kullanmadan güvenle devam ettik.

12.3–12.4 | parse mı, tryParse mı? Ve Türkçe virgül

`int.parse('üç')` (12.3)

`FormatException`: program çöker.

`int.tryParse('üç')` (12.3)

`null` döner; kullanıcıya düzgün hata mesajı gösterilir.

Türkçe virgül (12.4)

`double.tryParse('49,90')` → `null`

Çözüm: `girdi.replaceAll(',', '.')`

Kontrol sırası: `adet == null` → `adet <= 0` → `adet > 10` → kabul.

VS Code | 12.3–12.5 · Çökmeyen girdi

Dosyalar

`12_03_int_parse.dart` · `12_04_int_tryparse.dart` · `12_05_double_virgul.dart`
`12_06_dogrulama_do_while.dart` · `12_07_dogrulama_while_true.dart` **Birlikte yazalım**

1. 12_03 ve 12_04'te 3, üç, 0, 25, `2.5` girdilerini test edin.
2. 12_05'te `49,90`, 100, `-5`, `abc` deneyin.
3. Geçerli yaş girilene kadar soran `do-while` ile `while (true)` sürümlerini karşılaştırın.

Sorgu

Bir soru en az bir kez sorulmak zorundaysa neden `do-while` uygundur?

12_04_int_tryparse.dart *girilen → çıktı*

```
3    → Sepete 3 adet eklendi.
üç   → "üç" geçerli bir sayı değil.
0    → Adet en az 1 olmalı.
25   → Tek seferde en fazla 10 adet alınabilir.
2.5  → "2.5" geçerli bir sayı değil.
```

12_04_int_tryparse.dart *girilen → çıktı*

```
3    → Sepete 3 adet eklendi.
üç   → "üç" geçerli bir sayı değil.
0    → Adet en az 1 olmalı.
25   → Tek seferde en fazla 10 adet alınabilir.
2.5  → "2.5" geçerli bir sayı değil.
```

12_05_double_virgul.dart *girilen → çıktı*

```
49,90 → KDV dahil fiyat: 59.88 TL
100   → KDV dahil fiyat: 120.00 TL
-5    → Geçerli bir fiyat girin.
abc   → Geçerli bir fiyat girin.
```

12_03'te üç girilirse: `FormatException: Invalid radix-10 number ...` ve program durur.

12_06 do-while

```
Yaşınız (13-120): on sekiz
Lütfen yalnızca rakam girin.
Yaşınız (13-120): 8
Yaş 13 ile 120 arasında olmalı.
Yaşınız (13-120): 19
Kayıt tamamlandı. Yaşınız: 19
```

12_07 while (true) + break

```
Adet (1-10): 15
Geçersiz giriş, tekrar deneyin.
Adet (1-10): 4
Sepete 4 adet eklendi.
```

12_06 do-while

```
Yaşınız (13-120): on sekiz
Lütfen yalnızca rakam girin.
Yaşınız (13-120): 8
Yaş 13 ile 120 arasında olmalı.
Yaşınız (13-120): 19
Kayıt tamamlandı. Yaşınız: 19
```

12_07 while (true) + break

```
Adet (1-10): 15
Geçersiz giriş, tekrar deneyin.
Adet (1-10): 4
Sepete 4 adet eklendi.
```

İki yol, aynı amaç

do-while: soru **en az bir kez** sorulur, koşul sonda. while (true): çıkış **ortada**, geçerli değer gelince `break`. İkisinde de kullanıcı hatalı girdide **tekrar sorulur**, program çökmez.

12.5–12.8 | Kullanıcı girdisi: son kontrol listesi

1. Girdiyi güvenle alın: `readLineSync() ?? "`.
2. Boşlukları temizleyin: `trim()`.
3. Gerekirse `toLowerCase()` (12.6) veya `replaceAll(',', '.')` (12.4).
4. `int.tryParse` / `double.tryParse` ile dönüştürün.
5. `null`, aralık ve iş kurallarını test edin (12.7).
6. Geçersiz girdide **çökmeden** uyarın / yeniden sorun (12.5).
7. Türkçe karakter sorununda `chcp 65001` ve `readLineSync(encoding: utf8)` (12.8).

VS Code | 12.6–12.8 · Gerçek menü girdileri

Dosyalar

`12_08_evet_hayir.dart` · `12_09_iki_sayi.dart` · `12_10_utf8_girdi.dart` **Birlikte yazalım**

1. Evet / hayır sorusunda `E`, `evet`, `HAYIR`, `belki` deneyin.
2. İki sayıyı doğrulayıp işlem yapın; ikinci sayıyı `0` verin.
3. Türkçe karakter sorunu varsa terminal kodlamasını değiştirin.

Sorgu

`toLowerCase()` Türkçe `İ` / `ı` gibi karakterlerde neden dikkat gerektirir?

12_08 *girilen → çıktı*

```
E     → Bildirimler açıldı.
evet  → Bildirimler açıldı.
HAYIR → Bildirimler kapalı. Ayarlardan daha
sonra açabilirsiniz.
belki → Cevap anlaşılamadı; bildirimler
kapalı bırakıldı.
```

12_08 *girilen → çıktı*

```
E     → Bildirimler açıldı.
evet  → Bildirimler açıldı.
HAYIR → Bildirimler kapalı. Ayarlardan daha
sonra açabilirsiniz.
belki → Cevap anlaşılamadı; bildirimler
kapalı bırakıldı.
```

Neden `'hayir'` kolu var?

`'HAYIR'.toLowerCase()` sonucu `'hayir'` olur, `'hayır'` değil. Dart dönüşümü dilden bağımsız yapar.

12_09 (7 ve 2)

```
Birinci sayı: 7
İkinci sayı: 2
Toplam : 9.0
Fark   : 5.0
Çarpım : 14.0
Bölüm  : 3.50
```

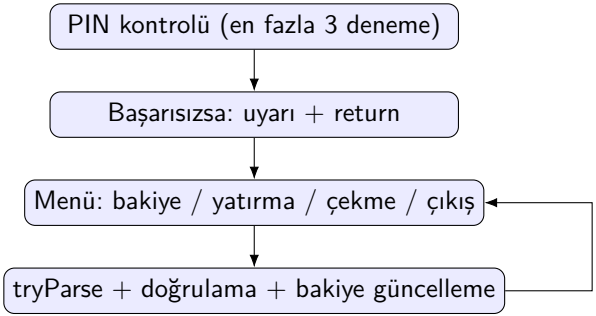
İkinci sayı 0 ise:
`Bölüm : Tanımsız (sıfıra bölme)`

13 | Hepsini Birleştirelim

Cep Cüzdan uygulaması

Ders notu: Bölüm 13 · Dosyalar: 13_01

13 | Uygulama: Cep Cüzdan



Bir arada: koşul (3), üçlü operatör (4), `switch` (5), kısa devre ve atama (6–7), `while` (9), `break` ve etiket (11), kullanıcı girdisi (12).

VS Code | 13 · Cep Cüzdan: canlı geliştirme

Terminalden çalıştırın

```
dart run 13_01_cep_cuzdan.dart
```

1. **Adım 1:** PIN için 3 deneme ve erken çıkış (`return`).
2. **Adım 2:** `menu: while (true)` + seçim için `switch`.
3. **Adım 3:** Bakiye göster, para yatır, para çek, çık (`break menu;`).
4. **Adım 4:** Hatalı sayı, `99,90` ve yetersiz bakiye senaryolarını test edin.

Sorgu

`break menu;` yerine yalnızca `break;` yazarsanız ne olur?

13_01_cep_cuzdan.dart *kullanıcının yaptığı → programın cevabı*

```
PIN 1111          → Hatalı PIN. Kalan hak: 2
PIN 2580          → Giriş başarılı.
Seçim 1           → Bakiyeniz: 1250.00 TL
Seçim 2, 250      → 250.00 TL yatırıldı. Yeni bakiye: 1500.00 TL
Seçim 3, 2000     → Yetersiz bakiye. Mevcut: 1500.00 TL
Seçim 3, 99,90    → 99.90 TL çekildi. Yeni bakiye: 1400.10 TL
Seçim 5           → Geçersiz seçim: "5"
Seçim 0           → Bu oturumda 2 işlem yaptınız. Güle güle!
```

13_01_cep_cuzdan.dart *kullanıcının yaptığı → programın cevabı*

```
PIN 1111          → Hatalı PIN. Kalan hak: 2
PIN 2580          → Giriş başarılı.
Seçim 1           → Bakiyeniz: 1250.00 TL
Seçim 2, 250      → 250.00 TL yatırıldı. Yeni bakiye: 1500.00 TL
Seçim 3, 2000     → Yetersiz bakiye. Mevcut: 1500.00 TL
Seçim 3, 99,90    → 99.90 TL çekildi. Yeni bakiye: 1400.10 TL
Seçim 5           → Geçersiz seçim: "5"
Seçim 0           → Bu oturumda 2 işlem yaptınız. Güle güle!
```

Üç kez hatalı PIN:

```
Hatalı PIN. Kalan hak: 2
Hatalı PIN. Kalan hak: 1
Hatalı PIN.
Çok fazla hatalı deneme. Hesabınız
geçici olarak kilitlendi.
```

Yalnızca `break;` olsaydı

Yalnızca switch biter; menü sonsuza kadar döner (Bölüm 11.7).

13 | Mini geliştirme görevi: Para transferi

Cep Cüzdan – yeni menü seçeneği

4) Para transferi ekleyin.

1. Alıcı adını ve transfer tutarını isteyin.
2. Tutarı `double.tryParse` ile doğrulayın.
3. Tutar bakiyeyi aşıyorsa uyarı verin.
4. Yeterliyse bakiyeden düşüp onay mesajı yazın: *"Ayşe adlı kişiye 50.00 TL gönderildi."*

Tartışma: Tek bir hata durumu bile kontrol edilmezse ne olabilir?

14 | Örnek Problem

Aynı probleme farklı doğru çözümler

Ders notu: Bölüm 14 · Dosyalar: `14_01` – `14_03`

14 | Problem: not ve harf notu hesaplama

Girdi ve hesap

Vize: 0–100, final: 0–100
Ortalama = `(vize * 40 + finalNotu * 60) / 100`

Ortalama	Harf
≥ 90	AA
≥ 85	BA
≥ 80	BB
≥ 75	CB

Ortalama	Harf
≥ 70	CC
≥ 60	DC
≥ 50	DD
< 50	FF

Final barajı

Final notu 50'nin altındaysa ortalama ne olursa olsun **FF**. Not: `final` bir anahtar kelimedir; değişken adı `finalNotu`.

14 | Sıra sizde: çözümden önce 15 dakika

1. `stdin` ile iki notu alıp `int.tryParse` ile kontrol edin.
2. Aralık dışı giriş varsa hata yazıp `return;` ile bitirin.
3. Ortalamayı hesaplayın ve iki ondalık gösterin.
4. Harf notunu ve **GEÇTİ / KALDI** sonucunu yazdırın.

Önce test senaryolarını yazın

(70, 80) → CB; (95, 45) → final barajı FF; (100, 90) → AA; (metin, 80) → geçersiz girdi.

14 | Aynı problem için üç doğru çözüm

Dosya	Yaklaşım	Ne zaman daha anlaşılır?
`14_01_not_if`	`if / else if`	Koşulları satır satır takip etme
`14_02_not_switch`	İlişkisel desenli `switch` ifadesi	Aralıkları tablo gibi ifade etme
`14_03_not_when`	`switch` + `when`	Final barajını desenin koruma koşuluna taşıma

Önemli: İyi çözüm yalnızca kısa olan değil; hata denetimi doğru, kenar durumları düşünülmüş ve okunabilir olandır.

VS Code | 14 · Not hesaplama: üç yaklaşım

Terminalden çalıştırın

`dart run 14_01_not_if.dart` (diğerleri de aynı şekilde)

1. Aynı dört test senaryosunu üç programda da uygulayın.
2. Final barajının hangi satırda kontrol edildiğini bulun.
3. Harf ölçeği değişse (ör. DC sınırı 65 olsa) hangi çözümü değiştirmek daha kolay?

Sorgu

Testler aynıysa farklı sözdizimleri neden aynı sonucu üretmeli?

Dönüş | 14 · Beklenen çıktılar ve final barajı

```
Vize notu (0-100): 70
Final notu (0-100): 80
Ortalama: 76.00
Harf notu: CB → GEÇTİ
```

```
Vize notu (0-100): 100
Final notu (0-100): 90
Ortalama: 94.00
Harf notu: AA → GEÇTİ
```

```
Vize notu (0-100): 95
Final notu (0-100): 45
Ortalama: 65.00
Harf notu: FF → KALDI (final barajı)
```

```
Vize notu (0-100): yetmiş
Final notu (0-100): 80
Notlar 0 ile 100 arasında tam sayı olmalı.
```

Dönüş | 14 · Beklenen çıktılar ve final barajı

```
Vize notu (0-100): 70
Final notu (0-100): 80
Ortalama: 76.00
Harf notu: CB → GEÇTİ
```

```
Vize notu (0-100): 100
Final notu (0-100): 90
Ortalama: 94.00
Harf notu: AA → GEÇTİ
```

```
Vize notu (0-100): 95
Final notu (0-100): 45
Ortalama: 65.00
Harf notu: FF → KALDI (final barajı)
```

```
Vize notu (0-100): yetmiş
Final notu (0-100): 80
Notlar 0 ile 100 arasında tam sayı olmalı.
```

Çözüm	Final barajı nerede kontrol ediliyor?
14_01	`else if` zincirinin **ilk** koşulu: `if (finalNotu < 50)`
14_02	Switch ifadesinden **sonra** ayrı bir `if` ile harf `FF` yapılıyor
14_03	Switch ifadesinin **ilk kolu**: `_ when barajAltinda => 'FF'`

15–18 | Pekiştirme ve Ödev

Özet, kendinizi test edin, Ödev 1, kaynaklar

Ders notu: Bölüm 15–18 · Dosyalar: –

15 | Hızlı özet: doğru araç, doğru problem

İhtiyaç	Araç
Karar / değer üretme	`if`, `? :`, `switch`
Tekrar sayısı belli	`for`, `for-in`
Koşula bağlı / en az bir tur	`while`, `do-while`
Akışı kes / turu atla	`break`, `continue`, label
Klavye girdisi	`stdin.readLineSync()`
Güvenli sayıya dönüşüm	`int.tryParse`, `double.tryParse`

Ana beceri: Doğru sonucu üreten, okunabilir ve hata durumlarını yöneten kod yazmak.

16 | Kendinizi test edin: Kararlar

S1 `puan = 75`: Ayrı `if` ile `else if` aynı sayıda çıktı mı üretir?

S2 `int sayi = 5; if (sayi) {...}` neden derlenmez?

S5 `ad == null || ad.isEmpty` neden güvenlidir; tersi neden sorunludur?

S6 `switch` ifadesinde `_ =>` yoksa ne olabilir?

16 | Kendinizi test edin: Kararlar

S1 `puan = 75`: Ayrı `if` ile `else if` aynı sayıda çıktı mı üretir?

S2 `int sayi = 5; if (sayi) {...}` neden derlenmez?

S5 `ad == null || ad.isEmpty` neden güvenlidir; tersi neden sorunludur?

S6 `switch` ifadesinde `_ =>` yoksa ne olabilir?

Kısa cevaplar

S1 Hayır: ayrı if'ler A ve B, zincir yalnızca A. **S2** Koşul `bool` olmalı. **S5** Kısa devre: null ise sağ taraf hiç çalışmaz. **S6** Eksiksiz değilse derleme hatası.

Ayrıntılı cevaplar: **Ders notu Bölüm 16**.

16 | Kendinizi test edin: Döngüler ve girdi

S8 Koşul başlangıçta yanlışsa `while` / `do-while` kaç tur döner?

S10 Döngü içindeki `switch`'te `break;` neden menüyü kapatmayabilir?

S11 `readLineSync()` neden `String?` üretir?

S12 `double.tryParse('3,5')` sonucu nedir; nasıl düzeltirsiniz?

16 | Kendinizi test edin: Döngüler ve girdi

S8 Koşul başlangıçta yanlışsa `while` / `do-while` kaç tur döner?

S10 Döngü içindeki `switch`'te `break;` neden menüyü kapatmayabilir?

S11 `readLineSync()` neden `String?` üretir?

S12 `double.tryParse('3,5')` sonucu nedir; nasıl düzeltirsiniz?

Kısa cevaplar

S8 0 ve 1. **S10** `break` en yakın switch'ten çıkar; etiket veya bayrak gerekir. **S11** Girdi akışı kapanabilir. **S12** `null`; `replaceAll(',', '.')`.

Notta 12 soru var; **kod yazmadan önce** çözün.

17 | Ödev 1: Teslim ve kapsam

AI Seviye 0

Ödev yapay zekâ kullanmadan yapılacak; 3. haftanın ilk 15 dakikasında çözümler sınıfta tartışılacak.

- **Zorunlu:** 1–6. sorular; **ek puan:** 7–8. sorular.
- Her soru ayrı `odev1_s1.dart`, `odev1_s2.dart`, ... dosyasında.
- Girdiler `tryParse` ile kontrol edilecek; hatalı girdide çökme olmayacak.
- Dosyaları biçimlendirin; **Problems** panelinde hata kalmasın.
- Son tarih ve teslim yöntemi Classroom'daki ödev sayfasında.

17 | Ödev 1: Zorunlu sorular I

1. **Pil tasarruf simülasyonu:** `while`; 20 üzeri dakikada 4, 20 ve altı 2 puan azalma; tasarruf uyarısını bir kez yaz.
2. **Kargo ücreti:** premium / 300 TL sınırı; bölgeye göre `switch` ifadesiyle 29,90 / 39,90 / 59,90 TL.
3. **Asal sayı:** `for` + `break`; bölenleri $d \times d \leq n$ sınırına kadar dene.

Hepsinde: Geçersiz girdi ne olacak? Sınır değerlerde ne çıkacak? Beklenen çıktılar ders notunun 17. bölümünde.

17 | Ödev 1: Zorunlu sorular II ve ek puan

4. **Ayın gün sayısı:** `switch` ifadesi + artık yıl (4'e bölünür ve 100'e bölünmez, ya da 400'e bölünür).
5. **Birim dönüştürücü:** tekrarlayan menü, km/mil ve °C/°F; `0` ile etiketli `break`.
6. **Sayı tahmini:** 1–100 rastgele hedef, en fazla 7 hak, `do-while`; geçersiz giriş hak harcamaz.

Ek puan

7) 5 yıldızlık değerlendirme göstergesi. 8) Şifre güç göstergesi (`for-in`, `if`, `switch`).

18 | Kaynaklar ve gelecek hafta

Dart resmi belgeleri

- `https://dart.dev/language/branches`
- `https://dart.dev/language/loops`
- `https://dart.dev/language/operators`
- `https://dart.dev/language/patterns`
- `https://api.dart.dev/`

Hafta 3 | Fonksiyonlar ve Koleksiyonlar

Fonksiyonlar, parametreler, listeler, Set ve Map. Derse gelmeden **Ödev 1**'i ve ders notunun **16. bölümünü** tamamlayın.

Önce tahmin et.
Sonra çalıştır.
Nedenini açıkla.

Sonraki adım: **Ödev 1** + **Hafta 3** hazırlığı